

Software Requirement Specifications

Visually Essential Social (VES)



Submitted By

Muhammad Adil Nadeem
S25BINFT7M04028

Submitted To

Prof. Dr. Musarat Karim

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Table of Contents

1. Introduction.....	4
1.1. Purpose.....	4
1.2. Scope.....	4
1.3. Product Perspective.....	5
1.4. User Characteristics.....	5
1.5. Similar Apps and Systems / Literature Review.....	6
1.6. Proposed Technologies.....	6
2. Requirements.....	7
2.1. Functional Requirements.....	7
2.1.1. User Registration (Sign Up).....	7
2.1.2. User Login.....	7
2.1.3. Create Post.....	8
2.1.4. Privacy Controls.....	8
2.1.5. Friend Management.....	9
2.1.6. Groups and Community.....	9
2.1.7. Analytics Dashboard.....	10
2.1.8. Admin Moderation.....	10
2.2. Non-Functional Requirements.....	11
3. Use Cases and Flow of Processes.....	12
3.1. Use Case 1 – User Registration.....	12
3.2. Use Case 2 – Create and Share Post.....	13
3.3. Use Case 3 – Admin Moderates Content.....	14
3.4. Entity Relationship (ER) Diagram.....	15
3.5. Data Flow Diagram (DFD).....	16
4. References.....	17

1. Introduction

Visually Essential Social (VES) is a web-based social media platform developed as a Final Year Project at the Islamia University of Bahawalpur. The system is designed using modern web technologies — React.js on the frontend, Node.js with Express.js on the backend, and Firebase or MySQL as the database. VES addresses a real gap in the Pakistani digital space: the absence of a localized, culturally relevant, bilingual social media platform that gives users genuine control over their privacy. The platform allows users to create profiles, share content in Urdu and English, interact with friends, join community groups, and track engagement through an analytics dashboard.

1.1. Purpose

The purpose of this SRS document is to formally define all requirements for the Visually Essential Social (VES) platform. It serves as the main reference for the development team, the project supervisor, and academic evaluators throughout the project lifecycle. The document specifies what the system must do, how it must behave, and what quality standards it must meet.

- Build a responsive single-page web application using React.js and Tailwind CSS.
- Develop a RESTful API backend with Node.js and Express.js to handle all business logic and data operations.
- Implement Firebase or MySQL as the primary database for storing users, posts, groups, and notifications.
- Provide multilingual support in both Urdu and English with proper RTL rendering.
- Support role-based access for General Users, Business Users, Group Managers, and Administrators.

1.2. Scope

The VES platform will be developed as a full-stack web application. The scope of development covers the following modules:

- User Module: Registration, login, profile management, privacy settings, friend requests, and notifications.
- Content Module: Post creation with text, images, and video; likes, comments, shares, and feed management.
- Groups Module: Group creation, membership management, and group-based content sharing.
- Analytics Module: Engagement tracking, post reach, and profile growth data for business users.
- Admin Module: Full user management, content moderation, and system-level statistics.

Out of Scope: Native mobile applications (iOS/Android), integration with external social media platforms, advanced AI-based recommendation engines, and real-time chat are not included in version 1.0.

1.3. Product Perspective

VES follows a standard three-tier web architecture. The React.js frontend communicates with the Node.js/Express.js backend through RESTful API calls. The backend processes requests, applies business logic, handles authentication, and queries the database. Firebase (cloud-hosted NoSQL) or MySQL (relational) stores all platform data including users, posts, comments, friendships, groups, and notifications. Tailwind CSS ensures a modern, responsive interface. JavaScript is the unified language across both frontend and backend, which simplifies development and maintenance.

Fig 1: VES Three-Tier System Architecture

CLIENT LAYER	SERVER LAYER	DATABASE LAYER
React.js Frontend HTML5 / CSS3 Tailwind CSS JavaScript ES6+	Node.js + Express.js REST API Routes Middleware / Auth Business Logic	Firebase / MySQL Data Storage User Collections Posts & Groups
Browser Client	Express Router	Database
Chrome / Firefox Safari / Edge Mobile Browser	Auth Routes Post Routes Group Routes	Users Posts Notifications

1.4. User Characteristics

The following user roles interact with VES, each with different access levels and technical requirements:

User Role	Description	Technical Expertise
General User	Registers and logs in to browse posts, share content, interact with friends, and manage privacy settings through the React.js interface.	Low to Moderate (basic browser usage)
Business User	Manages a business profile, creates promotional posts, and tracks audience engagement through the analytics dashboard.	Moderate (basic computer literacy)
Group Manager	Creates and manages public or private groups for educational or community purposes, controls membership and group content.	Moderate
Administrator	Has full access to manage all users, moderate reported content, and view system-level statistics via the admin panel.	High (IT background required)

1.5. Similar Apps and Systems / Literature Review

Several existing social media platforms and academic studies informed the design choices of VES:

- Facebook: The dominant global social platform with strong content sharing and group features. Limitation: not localized for Pakistani culture; privacy concerns are well-documented.
- Instagram: Visually focused platform widely used in Pakistan. Limitation: heavily algorithm-driven feed; limited bilingual or RTL support for Urdu content.
- Twitter/X: Text-focused platform. Limitation: no structured group or community feature; content discovery is driven by trending topics, not personal connections.
- Local Gap: No existing platform specifically targets Pakistani social media users with Urdu-first design, local privacy controls, and open community group features.
- MERN/React Stack Research: Multiple academic studies (React Documentation, Meta 2024; Node.js Foundation 2024) demonstrate the suitability of this stack for building scalable, interactive web applications with fast development cycles.

VES addresses these gaps by delivering a culturally adapted, bilingual, privacy-focused social platform built on proven web technologies for a local Pakistani audience.

1.6. Proposed Technologies

Category	Technology	Purpose
Frontend	React.js, HTML5, CSS3, Tailwind CSS, JavaScript	User interface, responsive design, dynamic components
Backend	Node.js, Express.js	REST API server, routing, middleware, business logic
Database	Firebase or MySQL	Data storage for users, posts, groups, notifications
Authentication	Session-based Auth / JWT	Secure user session management
Analytics	Google Analytics	User behavior tracking and engagement metrics
Version Control	Git, GitHub	Source code management and team collaboration
Development IDE	Visual Studio Code	Primary development environment

2. Requirements

The VES platform must handle the following core operations: user account management via authenticated API routes; post creation and feed delivery rendered dynamically by React.js components; cart-style state management for draft posts; social interaction processing through Express.js endpoints connected to the database; real-time notification delivery; group management with role-based access; analytics reporting; and an admin panel for full platform oversight. All user interfaces are built with React.js and Tailwind CSS to ensure a responsive, modern experience across all devices.

2.1. Functional Requirements

2.1.1. User Registration (Sign Up)

Name	FR001
Purpose	Allow a new user to create an account on the VES platform to access personalized features.
User(s)	General User, Business User, Group Manager
Input	Full Name: Real name of the user. Email Address: Valid and unique email stored in the database. Password: Minimum 8 characters, at least 1 number and 1 special character, hashed before storage. Phone Number: Pakistani mobile number. Role: Selected from General User, Business User, or Group Manager.
Output	New user record created in the database. Verification email sent. User redirected to login page after verification.

2.1.2. User Login

Name	FR002
Purpose	Authenticate a registered user and establish a secure session for platform access.
User(s)	General User, Business User, Group Manager, Administrator
Input	Email: Registered email address. Password: User password, verified against the stored hash.
Output	Secure session established. User redirected to their role-specific React.js dashboard. Failed login returns an error message with guidance.

2.1.3. Create Post

Name	FR003
Purpose	Allow logged-in users to create and publish posts containing text, images, or video content.
User(s)	General User, Business User
Input	Post Content: Text written in Urdu or English. Media: Optional image or video file upload. Visibility: Selected from Public, Friends Only, or Private. Language: Urdu or English toggle.
Output	New post record created in the database. Post appears in the user's profile and in the feeds of relevant friends. Engagement counters initialized to zero.

2.1.4. Privacy Controls

Name	FR004
Purpose	Allow users to manage who can see their posts and profile information.
User(s)	General User, Business User
Input	Post Visibility Setting: Public, Friends Only, or Private chosen before publishing. Profile Visibility Setting: Configured from account settings. Block Action: User ID of the person to be blocked.
Output	Post visibility stored in the database and enforced on every feed query. Profile visibility applied across all views. Blocked user is prevented from seeing the blocking user's content or profile.

2.1.5. Friend Management

Name	FR005
Purpose	Allow users to send friend requests, manage connections, and receive friend suggestions.
User(s)	General User
Input	Friend Request: Target user ID sent through the API. Accept or Decline Action: Response from the receiving user. Search Query: Optional keyword to find users.
Output	Friend request record created in the database. Upon acceptance, a bidirectional connection is established. Friend suggestion list updated based on shared interests and mutual friends. Notifications sent to both users.

2.1.6. Groups and Community

Name	FR006
Purpose	Allow users to create, join, and manage groups for educational or community purposes.
User(s)	Group Manager, General User
Input	Group Name and Description: Basic group information. Group Type: Public or Private. Join Request: User ID requesting membership. Member Management Action: Add, remove, or promote member.
Output	Group record created in the database. Membership updated based on join requests and admin decisions. Group feed shows posts from all group members visible only within the group.

2.1.7. Analytics Dashboard

Name	FR007
Purpose	Provide users and business accounts with engagement data for their posts and profiles.
User(s)	General User, Business User
Input	Post ID or Profile ID: The content for which analytics are requested. Date Range: Optional filter for the time period.

Output	Analytics data returned including total views, likes, comments, shares, profile visits, and follower growth trend. Data displayed as charts and numbers on the React.js dashboard.
---------------	--

2.1.8. Admin Moderation

Name	FR008
Purpose	Allow administrators to manage user accounts, review reported content, and monitor platform activity.
User(s)	Administrator
Input	Report Queue: List of all pending reports submitted by users. Admin Action: Remove content, issue warning to user, suspend account, or dismiss report. User Search Query: For finding and managing specific accounts.
Output	Selected admin action applied immediately to the database. User notified of warnings or suspensions via in-app notification. Report status updated to Resolved. System statistics dashboard refreshed.

2.2. Non-Functional Requirements

Category	Req. ID	Specification
Performance	NFR001	All API responses from the Express.js server must be returned within 3 seconds under normal load conditions.
Reliability	NFR002	The system must maintain a minimum uptime of 99% during the testing and evaluation period. Data must not be lost in case of unexpected failure.
Security	NFR003	All passwords must be hashed before storage. User sessions must expire after inactivity. All communication must use HTTPS. Users must not be able to access other users' private data.
Usability	NFR004	A new user must be able to register and publish their first post within 5 minutes. All major features must be reachable within 2–3 clicks. Urdu RTL rendering must be correct.
Scalability	NFR005	The Node.js/Express backend must support at least 500 concurrent users without performance degradation.
Maintainability	NFR006	The codebase must follow MVC pattern, include inline documentation, and be version-controlled on GitHub with meaningful commit messages.
Compatibility	NFR007	The React.js frontend must render correctly on Chrome, Firefox, Edge, and Safari on desktop and tablet devices.
Data Integrity	NFR008	All database transactions involving posts and user data must be complete. Partial writes must be prevented.

3. Use Cases and Flow of Processes

The use cases below describe the primary interactions between system actors and the VES platform. All use cases are implemented as React.js frontend flows backed by Express.js API endpoints and database operations. The four main actors are the General User, Business User, Group Manager, and Administrator. Each use case links directly to the functional requirements defined in Section 2.

System Level Use Case Diagram – Visually Essential Social (VES)

General User	Business User	Group Manager	Administrator
Register / Login	Register / Login	Register / Login	Admin Login
Create & Share Post	Create & Promote Post	Create & Manage Group	Manage User Accounts
Like / Comment / Share	View Analytics	Post in Group	Moderate Reported Content
Send Friend Request	Track Audience Growth	Add / Remove Members	View System Statistics
Manage Privacy Settings	–	Promote Members	–
Block / Report User	–	–	–

Fig 2: System Level Use Case Diagram

3.1. Use Case 1 – User Registration

ID	UC001
Name	User Registration
Description	This use case describes the process by which a new user registers on the VES platform through the React.js registration form, submitting data to the Express.js API which stores it in the database.
Requirement(s)	FR001
Actor(s)	General User, Business User, Group Manager
Precondition	User must not already have an account with the provided email address in the database.
Postcondition	A new user record is created in the database. A verification email is sent. User can log in after clicking the verification link.
Basic Flow	Basic Flow: 1. User navigates to the /register React.js page. 2. User fills in the registration form with name, email, password, phone, and role. 3. React.js validates inputs on the client side (email format, password strength). 4. On submit, React.js sends POST /api/auth/register to the Express.js server. 5. Express.js validates the request body and checks for duplicate email. 6. Password is hashed and a new user record is saved to the database. 7. A verification email is sent to the registered address. 8. User clicks the verification link and is redirected to /login. Alternative Flow: A1. Administrator creates a user account directly from the admin panel. Exceptions: E1. Invalid email

format — server returns 400 with validation error; form displays inline message. E2.
Password too weak — client-side validation blocks submission. E3. Email already exists
— server returns 409 Conflict; user is prompted to log in instead.

3.2. Use Case 2 – Create and Share Post

ID	UC002
Name	Create and Share Post
Description	This use case describes how a logged-in user creates a new post with text, image, or video content using the React.js post editor, submits it through the Express.js API, and the post is saved to the database and published to relevant feeds.
Requirement(s)	FR003, FR004
Actor(s)	General User, Business User
Precondition	User must be logged in with a valid session. The user must have a complete profile.
Postcondition	A new post record is created in the database. The post appears in the user's profile and in the feeds of their friends. Engagement counters are initialized to zero.
Basic Flow	Basic Flow: 1. User clicks the 'Create Post' button on the React.js home page. 2. User enters text content and optionally uploads an image or video. 3. User selects the post language (Urdu or English) and visibility setting. 4. React.js validates the input and sends POST /api/posts with the session token. 5. Express.js validates the request, saves the post record to the database. 6. The post is pushed to the user's profile feed and to connected friends' feeds. 7. User receives a confirmation and the post appears immediately. Alternative Flow: A1. User edits a published post — React.js sends PUT /api/posts/:id; database record updated. A2. User deletes a post — React.js sends DELETE /api/posts/:id; post removed from all feeds. Exceptions: E1. Session expired — server returns 401; React.js redirects to /login. E2. Media file too large — server returns 413; React.js prompts user to upload a smaller file. E3. Database write fails — server returns 500; React.js shows a retry prompt.

3.3. Use Case 3 – Admin Moderates Content

ID	UC003
Name	Admin Moderates Reported Content
Description	This use case describes how an administrator reviews content reported by users and takes the appropriate moderation action. The React.js admin panel displays the report queue fetched from the database via the Express.js API.
Requirement(s)	FR008
Actor(s)	Administrator
Precondition	Admin must be logged in. At least one report must exist in the moderation queue in the database.
Postcondition	The reported content is either removed, the user is warned, or the report is dismissed based on the admin's decision. The database record is updated accordingly.
Basic Flow	Basic Flow: 1. Admin logs in and navigates to the moderation section of the admin panel. 2. React.js fetches GET /api/admin/reports from Express.js; queue displayed. 3. Admin reviews the reported content and the reason provided by the reporter. 4. Admin selects an action: Remove Content, Issue Warning, Suspend Account, or Dismiss Report. 5. React.js sends the selected action to the Express.js API. 6. Express.js applies the action, updates the database record, and notifies the affected user. 7. Report status is set

to Resolved and removed from the active queue. Alternative Flow: A1. Content was already deleted by the original poster before admin review — system automatically resolves the report. Exceptions: E1. Admin session expires — server returns 401; admin is redirected to the login page. E2. Content record not found in database — server returns 404; admin panel shows 'Content not available'.

3.4. Entity Relationship (ER) Diagram

The diagram below illustrates the main database entities for VES, their key attributes, and their relationships as implemented in Firebase or MySQL.

Fig 3: VES Database Entity Relationship Diagram

Entity	Key Attributes	Relationships
User	UserID (PK), Name, Email, PasswordHash, ProfilePhoto, Bio, Role, Language, CreatedAt	Creates Posts, Writes Comments, Sends/Receives Friend Requests, Joins Groups, Receives Notifications
Post	PostID (PK), UserID (FK), Content, MediaURL, MediaType, Visibility, Language, CreatedAt, UpdatedAt	Belongs to one User, Has many Comments and Likes, Can be Shared
Comment	CommentID (PK), PostID (FK), UserID (FK), Text, CreatedAt	Belongs to one Post, Written by one User
Like	LikeID (PK), PostID (FK), UserID (FK), CreatedAt	Given by one User on one Post
Friendship	FriendID (PK), UserID1 (FK), UserID2 (FK), Status, RequestedAt, AcceptedAt	Bidirectional link between two User records
Group	GroupID (PK), Name, Description, Type (Public/Private), CreatedByUserID (FK), CreatedAt	Created by one User, Has many GroupMembers and Group Posts
GroupMember	MemberID (PK), GroupID (FK), UserID (FK), Role, JoinedAt	Junction entity connecting User and Group
Notification	NotifID (PK), ReceiverUserID (FK), Type, ReferenceID, IsRead, CreatedAt	Sent to one User, triggered by Post, Comment, or Friend Request events
Report	ReportID (PK), ReporterUserID (FK), ContentType, ContentID, Reason, Status, CreatedAt	Filed by User, Reviewed by Administrator

Fig 3: VES Database Schema Diagram

3.5. Data Flow Diagram (DFD)

The activity and data flow diagrams below illustrate the main process flows in VES, showing how data moves between users, the React.js frontend, the Express.js backend, and the database.

Level 0 – Context Diagram (System Overview)

External Entity	Data Sent to VES	Data Received from VES
General User	Registration info, login credentials, posts, comments, likes, friend requests, privacy settings	Personalized feed, notifications, friend suggestions, search results, engagement counts
Business User	Posts, promotional content, analytics requests	Analytics reports, engagement metrics, follower growth data
Group Manager	Group creation data, member management actions, group posts	Updated group membership records, group activity feeds
Administrator	Moderation decisions, user management commands	Reports queue, system statistics, user account status updates

Level 1 – Internal System Processes

P#	Process	Input Data	Output Data
P1	User Authentication	Email and password credentials	Session token; access granted or denied
P2	Content Management	Post content, media files, edit/delete actions	Stored posts, updated feeds, confirmation
P3	Social Interaction	Like, comment, and share actions	Updated engagement counts, triggered notifications
P4	Friend Management	Friend requests, accept/decline responses	Updated connection list, friend suggestions, notifications
P5	Group Management	Group creation data, join requests, member actions	Updated membership records, group activity feeds
P6	Analytics Engine	Engagement events (likes, views, comments)	Analytics charts and reports for the dashboard
P7	Notification System	System events triggered by user actions	In-app notifications delivered to relevant users
P8	Admin Moderation	Submitted reports and admin decisions	Content status updates, account changes, moderation logs

Fig 4: VES Data Flow Diagram

4. References

1. IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, Jun. 1998.
2. Meta Platforms, Inc., "React.js Documentation," [Online]. Available: <https://react.dev>. [Accessed: 2025].
3. OpenJS Foundation, "Node.js Documentation," [Online]. Available: <https://nodejs.org/en/docs>. [Accessed: 2025].
4. OpenJS Foundation, "Express.js Documentation," [Online]. Available: <https://expressjs.com>. [Accessed: 2025].
5. Google Firebase, "Firebase Documentation," [Online]. Available: <https://firebase.google.com/docs>. [Accessed: 2025].
6. Oracle Corporation, "MySQL Documentation," [Online]. Available: <https://dev.mysql.com/doc>. [Accessed: 2025].
7. Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 2025].
8. Google LLC, "Google Analytics Documentation," [Online]. Available: <https://developers.google.com/analytics>. [Accessed: 2025].
9. GitHub Inc., "GitHub Documentation," [Online]. Available: <https://docs.github.com>. [Accessed: 2025].
10. S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80-83, Nov.-Dec. 2010.